



Práctica 2

PROCESOS E HILOS

1. Información importante

Todos los ejemplos de código se encuentran en <https://labdcc.fceia.unr.edu.ar/~jcaporalini/ips/lab02.zip> leer y ver atentamente.

Todos los ejercicios están pensados para ser ejecutados en linux (o macOS). Si usas windows, te recomiendo usar WSL (Windows Subsystem for Linux), una maquina virtual o github codespaces.

Manejo de procesos

Usualmente cuando ejecutamos un programa esperamos una ejecución sentencia a sentencia, de forma secuencial. Un ejemplo bastante claro y evidente de esto es el siguiente:

Figura 1: Código 1

```
#include <stdio.h>

int main(){
    printf("5\n");
    printf("4\n");
    printf("3\n");
    printf("2\n");
    printf("1\n");
    printf("0\n");
    printf("Hola Mundo!!!\n");
    return 0;
}
```

¿Qué pasa si cambiamos este código y simplemente agregamos una linea extra?

Figura 2: Código 2

```
#include <stdio.h>
#include <unistd.h>

int main(){

    fork();

    printf("5\n");
    printf("4\n");
    printf("3\n");
    printf("2\n");
    printf("1\n");
    printf("0\n");

    printf("Hola Mundo!!!\n");

    return 0;
}
```

Ejercicio 1. Ejecutá el código Código 2 varias veces (haceme caso, ejecutalo varias veces y vas a ver algo) y observá con atención la salida por pantalla. ¿Qué es lo que pasa?

¿Qué hay detrás de esa línea? ¿Qué es lo que está pasando?

Lo que sucede es que la función *fork* crea un nuevo proceso, podemos verlo si hacemos lo siguiente:

Figura 3: Código 3a

```
#include <stdio.h>
#include <unistd.h>

int main(){

    sleep(10);

    return 0;
}
```

Figura 4: Código 3b

```
#include <stdio.h>
#include <unistd.h>

int main(){

    fork();

    sleep(10);

    return 0;
}
```

Tip: ps

ps es una herramienta esencial que proporciona una “fotografía” estática de los procesos activos en el sistema. A diferencia de **top** o **htop**, no se actualiza en tiempo real. Muestra información como el ID del proceso (PID), uso de tiempo y el comando ejecutándose. Si querés ver más información podés buscar el manual de **ps** con el comando **man ps**.

Ejercicio 2. Ejecutá primero:

```
./codigo03a & ps
```

Y luego:

```
./codigo03b & ps
```

¿Cuál es la diferencia entre ambos comandos?

Si pudiste observar detenidamente, podrás notar que la diferencia entre ambas ejecuciones radica en que en la segunda ejecución aparece un proceso adicional, el cual no aparecía en la primera ejecución. En la segunda ejecución aparecen dos procesos, el proceso original y un nuevo proceso que es una copia del original creado a partir de la llamada a *fork*.

Ejercicio 3. Veamos cuan copia son uno del otro ejecutando el código 4 provisto dentro del laboratorio. Responder despues de ejecutar el codigo varias veces:

- a) ¿Se comporta igual?
- b) Luego del fork ¿Cuál es la diferencia en el arreglo?
- c) Luego del fork ¿Cuál es la diferencia en el archivo?
- d) Luego del fork ¿Cuál es la diferencia en los argumentos?
- e) Los cambios en un proceso se reflejan en el otro?

Como pudieron ver fork debería crear una copia del proceso hasta el momento de la invocación, los cambios realizados luego de esta son privados a cada proceso.

Ejercicio 4. ¿Qué pasa si dos pocesos leen por teclado y escriben en un archivo? Completar el `codigo05.c` del laboratorio.

En este “*copiado*” tenemos un proceso original o padre y otro proceso copia o hijo. Para poder distinguir entre uno y otro tenemos que mirar el valor de retorno de la función *fork* como se muestra en el siguiente código. Si es 0 estamos en el hijo, si es mayor a 0 estamos en el padre.

Figura 5: Código 6

```
#include <stdio.h>
#include <unistd.h>

int main(){

    int pid = fork();

    if (pid == 0) {
        printf("Soy el hijo\n");
    } else if (pid > 0) {
        printf("Soy el padre\n");
    } else {
        printf("Error al crear el hijo\n");
    }

    return 0;
}
```

En el caso del padre el valor obtenido al llamar a `fork` es el `pid` del hijo. Para obtener el `pid` de un proceso se puede usar la función `getpid()`, veamos un ejemplo.

Tip: pid

El PID (Process ID o identificador de proceso) en sistemas Unix/Linux es un número entero único asignado automáticamente por el núcleo (kernel) a cada proceso en ejecución. Permite al sistema operativo gestionar, monitorear y finalizar tareas de manera individual, reutilizando los números una vez que el proceso finaliza.

Figura 6: Código 7

```
#include <stdio.h>
#include <unistd.h>

int main(){

    printf("PID antes de fork: %d\n", getpid());

    int pid = fork();

    if (pid == 0) {
        printf("PID del hijo: %d\n", getpid());
    } else if (pid > 0) {
        printf("PID del padre: %d y el PID del hijo es: %d\n", getpid(), pid);
    } else {
        printf("Error al crear el hijo\n");
    }

    return 0;
}
```

Ejercicio 5. Implemente en `codigo08.c` que el hijo imprima el `pid` de su padre.

Ahora entendemos un poco más lo que provoca un `fork`. Su invocación crea un nuevo proceso. Hasta ahora una vez lanzado la comunicación entre padre e hijo es interrumpida. Existen varias formas de comunicar procesos¹ pero por lo pronto nos interesa una de las más elementales ¿Cómo saber cuando el hijo termina?

Existe una llamada a sistema que nos permite esperar a que termine el hijo: `wait`. Veamos como se usa.

Si no usamos `wait`, como en el código de la figura 9a, podemos ver que siempre imprime el padre y tiempo después el hijo. Eso se corrige haciendo que el padre espere al hijo con `wait`.

Ejercicio 6. ¿Qué pasa si se lanzan varios hijos desde el mismo padre? ¿Se espera a todos? ¿A uno?

¹Esperemos ver algunas

Figura 7: Código 9a

```
#include <stdio.h>
#include <unistd.h>

int main(){

    int pid = fork();

    if (pid == 0) {

        sleep(5);
        printf("> HIJO  <\n");

    } else if (pid > 0) {
        printf("> PADRE <\n");
    } else {
        printf("Error al crear el hijo\n");
    }

    return 0;
}
```

Figura 8: Código 9b

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){

    int pid = fork();

    if (pid == 0) {

        sleep(5);
        printf("> HIJO  <\n");

    } else if (pid > 0) {
        wait(NULL);
        printf("> PADRE <\n");
    } else {
        printf("Error al crear el hijo\n");
    }

    return 0;
}
```

También podemos usar `waitpid` para esperar a un hijo específico. Ver el código 10. Los argumentos para la función `waitpid` son:

- `pid` del hijo a esperar. Puede ponerse -1 para esperar a cualquier hijo comportandose como `wait`.
- `status`, Estado del hijo. Debe ser un puntero a donde se quiera guardar la información o `NULL` si no se quiere guardar la información.
- `options`: Opciones adicionales, utilizaremos 0 para no usar ninguna opción.

Ejercicio 7. ¿Qué está recuperando `WEXITSTATUS(status)`? Se puede modificar el código 10 para probar.

Figura 9: Código 10

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){

    int pid = fork();

    if (pid == 0) {

        sleep(5);
        printf("> HIJO  <\n");
        return 1;

    } else if (pid > 0) {

        int status;
        waitpid(pid, &status, 0);
        printf("> PADRE <\n");
        printf("Retorno del hijo (codigo exit): %d\n", WEXITSTATUS(status));
    } else {
        printf("Error al crear el hijo\n");
    }

    return 0;
}
```

Ejercicio 8. Ver el código 11 dentro del laboratorio y responder:

- a) ¿Cuántos procesos se ejecutan?
- b) ¿Son todos los hijos invocados por el mismo padre o por distintos?
- c) ¿Qué hace cada proceso?

Teniendo la posibilidad de crear nuevos procesos podemos sentirnos tentados a no solo ejecutar programas propios, sino que también ejecutar otros programas. Para hacer esto tenemos la familia de llamadas a sistema `exec`. Este grupo de funciones permite suplantar el código del programa actual por el de otro ejecutable. Si ejecutamos el siguiente código:

Tip: which

`which` es un comando que busca el path absoluto de un comando. Por ejemplo, `which uname` te dirá dónde está el ejecutable `uname`.

Figura 10: Código 12

```
#include <stdio.h>
#include <unistd.h>

int main(){

    char *path = "/usr/bin/uname";
    char *args[] = {"uname", NULL};

    printf("Llamando a uname con execv...\n");

    execv(path, args);

    printf("ESTO NUNCA SE EJECUTA SI EXECV FUNCIONA CORRECTAMENTE");
    return 1;
}
```

`execv` permite reemplazar el programa actual por otro a partir de su path absoluto. El primer argumento es la dirección absoluta del ejecutable dentro del sistema de archivos pero el segundo argumento (`char **`) es un array de strings que representan los argumentos que se le pasarán al programa siguiendo la convención de unix, primero se indica el nombre del programa y luego los argumentos. Para indicar el fin del array se coloca un `NULL`.

Por ejemplo si queremos ejecutar `ls -l -a -h` desde un programa en c, lo podemos hacer de la siguiente manera:

```
#include <stdio.h>
#include <unistd.h>

int main(){
    char *path = "/bin/ls";
    char *args[] = {"ls", "-l", "-a", "-h", NULL};
    execv(path, args);
    return 0;
}
```

También podemos ejecutar nuestros propios programas utilizando de la familia `exec`. Por ejemplo, podemos ejecutar el código 1 de la siguiente manera:

Figura 11: Código 13

```
#include <stdio.h>
#include <unistd.h>

int main(){
    char *path = "./codigo01";
    char *args[] = {"./codigo01", NULL};
    execv(path, args);
    return 0;
}
```

Este caso la ruta es relativa a la terminal que ejecuta el programa por lo que el código 13 debe ejecutarse desde el directorio que contiene el código 1. Si fuese necesario se podría agregar argumentos de la misma forma que antes.

Ejercicio 9. Es muy interesante combinar `fork` con `exec`. Creá un programan en el archivo `codigo14.c`, que invoque a 3 hijos, uno ejecutará `ls codigo01`, otro `pwd -L` y otro `date`, el padre debe esperarlos a los 3 a que finalicen, imprimirá el mensaje “Mis tres hijos terminaron su ejecución”.

Ejercicio 10. A partir de lo visto comentar las siguientes afirmaciones:

- a) `fork()` crea un nuevo proceso que es una copia exacta del proceso padre. Ambos procesos comparten memoria todo el tiempo al igual que archivos abiertos, por lo que si uno escribe en un archivo y el otro lo lee pueden ver lo escrito por el otro.
- b) `wait()` hace que el proceso padre espere a que termine el proceso sea o no hijo del padre.
- c) `execv()` permite ejecutar otro programa. Una vez que termina el programa llamado por las `execv` continua el programa que invocó a `execv`.

Manejo de hilos

Ya vimos en clases que los procesos son la forma que tiene los sistemas operativos de gestionar y garantizar aislamiento, administración de recursos y generar una abstracción de una ejecución. Sin embargo, esta forma de abstracción tiene un costo en cuanto a recursos y tiempos. Muchas veces es necesario romper el aislamiento para ahorrar recursos y permitir la comunicación de forma sencilla. Los hilos son una forma de lograr esto, en particular los hilos POSIX que son provistos por el sistema operativo. Para crear procesos usábamos la llamada a sistema `fork` pero para crear hilos usaremos una función provista por la librería POSIX que es `pthread_create`.

Figura 12: Código 15

```
#include <stdio.h>
#include <pthread.h>

void *funcion_hilo(void *arg){
    printf("Hola desde el hilo\n");
    return NULL;
}

int main(){
    pthread_t hilo; // Referencia al hilo

    // Crea un hilo que ejecutara la funcion funcion_hilo
    pthread_create(&hilo, NULL, funcion_hilo, NULL);

    printf("Hola desde el hilo principal\n");

    return 0;
}
```

Si se ejecuta varias veces el código 15 podemos ver que no siempre se imprime “Hola desde el hilo“. Esto se debe a que una vez que termina de ejecutarse la función `main`, el programa termina y por lo tanto todos los hilos que se hayan creado también terminan su ejecución. Para evitar esto podemos usar la función `pthread_join`.

Figura 13: Código 16

```
#include <stdio.h>
#include <pthread.h>

void *funcion_hilo(void *arg){
    printf("Hola desde el hilo\n");
    return NULL;
}

int main(){
    pthread_t hilo; // Referencia al hilo

    // Crea un hilo que ejecutara la funcion funcion_hilo
    pthread_create(&hilo, NULL, funcion_hilo, NULL);

    printf("Hola desde el hilo principal\n");

    // Espera a que el hilo termine
    pthread_join(hilo, NULL);

    return 0;
}
```

Esta función bloquea a quien la invoque hasta tanto el hilo referenciado como argumento termine su ejecución. Por lo tanto, la función `main` no terminará hasta que el hilo creado termine.

Ahora que tenemos un hilo funcionado pongámonos a analizar ambas funciones. cuando se invoca a `pthread_create` se le pasa como argumento un puntero a `pthread_t`, en el código 16 se declara una variable de este tipo y se le pasa la dirección de memoria como primer argumento, en el segundo argumento se pasa `NULL` que permite dar configuraciones particulares al hilo (no nos interesa en este curso más que el default), el tercer argumento es un puntero a la función que se ejecutará como hilo (en el caso del código 16 se pasa la función `funcion_hilo`, siempre debe tener esa signatura: `void * func(void *arg)`) y el último argumento es un puntero a un void (`void *`)² que se pasará como argumento a la función del hilo.

Un ejemplo de donde se le pasa a la función del hilo un argumento es el siguiente:

Figura 14: Código 17

```
#include <stdio.h>
#include <pthread.h>

void *funcion_hilo(void *arg){
    char *mensaje = (char *)arg;
    printf("Hola desde el %s\n", mensaje);
    return NULL;
}

int main(){
    pthread_t hilo1; // Referencia al hilo
    pthread_t hilo2; // Referencia al hilo

    char *mensaje[] = { "hilo 1", "hilo 2"};

    // Crea un hilo que ejecutara la funcion funcion_hilo
    pthread_create(&hilo1, NULL, funcion_hilo, (void *)mensaje[0]);
    pthread_create(&hilo2, NULL, funcion_hilo, (void *)mensaje[1]);

    printf("Hola desde el hilo principal\n");

    // Espera a que el hilo termine
    pthread_join(hilo1, NULL);
    pthread_join(hilo2, NULL);

    return 0;
}
```

En este caso tenemos dos variables de tipo `pthread_t` que guardan las referencias a los hilos que vamos a crear que son 2.

²Se que todavía están aprendiendo a usar punteros, por ahora (y hasta el segundo semestre) los argumentos que se le pasan a las funciones voy a dejar ayuda.

Ejercicio 11. Ejecutar el código 17 varias veces seguidas. ¿Qué conclusiones pueden sacar? Podes ayudarte con el script:

```
for _ in {1..10}; do echo "-----" ; ./codigo17; done
```

Como ya sabemos los hilos comparten espacio de memoria, es por eso que pueden acceder a las direcciones de memoria compartidas.

Ejercicio 12. Ver y ejecutar el código 18 dentro del laboratorio y responder:

- a) ¿Cuántos hilos se ejecutan?
- b) ¿Qué pasa con el archivo de lectura compartido?
- c) ¿Qué pasa con el archivo de escritura compartido?
- d) ¿Qué pasa con el arreglo compartido?

Ejercicio 13. Defina una función que dado el nombre de un archivo, cuente la cantidad de palabras en él. Luego diseñe un programa que por cada archivo pasado como argumento al programa cree un hilo para contar las palabras en él. Imprima el resultado cuando termina cada hilo. Usar el esqueleto en `codigo19.c`.